

AWS Cloud Project Documentation

QuickLoan

Scalable Web Application Deployment on Amazon Web Services

A complete cloud infrastructure project - VPC EC2 RDS ALB Auto Scaling S3

Prepared by: Milan Meena

Guided by: Mr. Lekharaj (Trainer)

Project Specifications

TECHNOLOGY STACK: PHP, Apache/Nginx, MySQL, AWS	CLOUD PLATFORM: Amazon Web Services (AWS)
REGION USED: Mumbai Region	DOCUMENT TYPE: Step-by-Step Technical Documentation

Table of Contents

- 1. Project Overview & Architecture
- 2. Technology Stack
- 3. Architecture Flow Diagram
- 4. Step 1-VPC & Networking Setup
- 5. Step 2-Security Groups Configuration
- 6. Step 3-EC2 Instance Launch
- 7. Step 4-Application Deployment (Nginx + PHP)
- 8. Step 5-S3 Bucket for Static Assets
- 9. Step 6-RDS MySQL Database Setup
- 10. Step 7-Domain Configuration (No-IP)
- 11. Step 8-AMI Creation
- 12. Step 9-Application Load Balancer & Target Group
- 13. Step 10-Auto Scaling Group Configuration
- 14. Step 11-End-to-End Verification
- 15. Roles & Responsibilities

1. Project Overview & Architecture

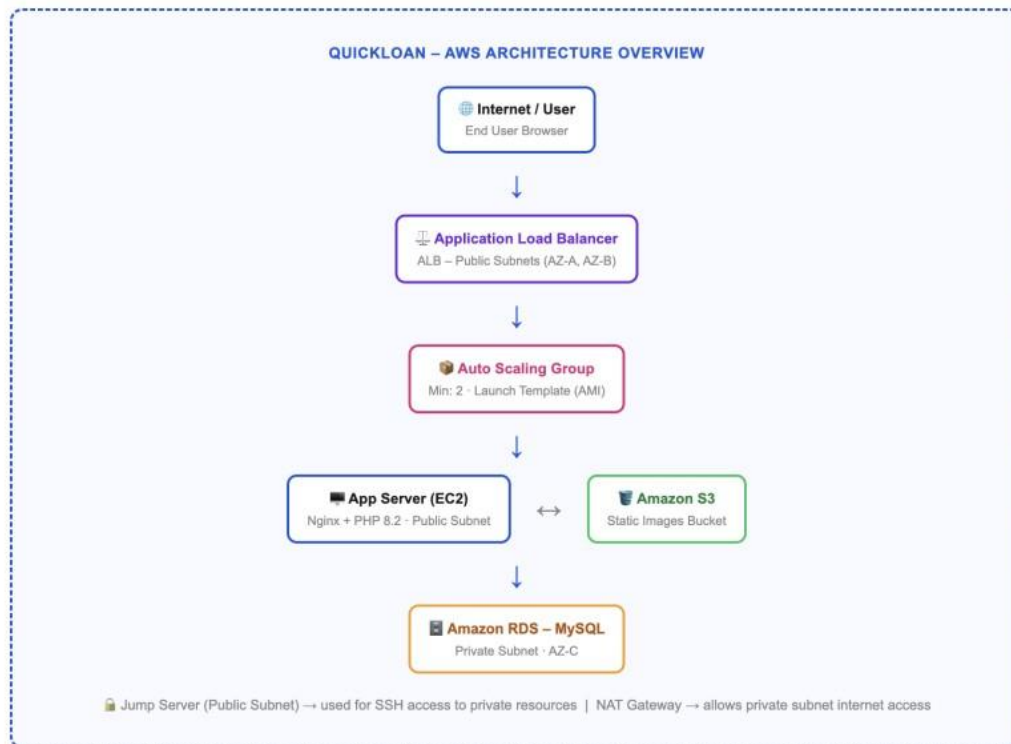
QuickLoan is a cloud-hosted loan application web platform built with a PHP backend and HTML/CSS/JavaScript frontend. The project demonstrates a production-grade AWS infrastructure setup encompassing networking, compute, storage, database, auto scaling, and load balancing - following industry best practices for high availability and fault tolerance.

The application was deployed inside a custom Virtual Private Cloud (VPC) with properly segmented public and private subnets across multiple Availability Zones. Traffic flows through an Application Load Balancer (ALB) to an Auto Scaling Group of EC2 instances, which communicate with a managed RDS MySQL database in a private subnet. Static assets (images) are served from Amazon S3.

2. Technology Stack

LAYER	TECHNOLOGY/SERVICE	PURPOSE
Frontend	HTML, CSS, JavaScript	User interface of the QuickLoan web app
Backend	PHP 8.2, php-fpm, phpmysqlnd	Server-side application logic
Web Server	Nginx	HTTP request handling and routing
Database	Amazon RDS-MySQL (MariaDB client)	Persistent storage of loan applications
Object Storage	Amazon S3	Hosting static image assets
Networking	VPC, Subnets, IGW, NAT Gateway, Route Tables	Isolated, secure cloud network
Compute	Amazon EC2 (Amazon Linux 2023)	Application, jump, and database servers
Scaling	Auto Scaling Group + Launch Template (AMI)	Horizontal scaling based on demand
Load Balancing	Application Load Balancer (ALB)	Traffic distribution across EC2 instances
DNS/Domain	No-IP (quickloan.hopto.org)	Free dynamic DNS for application access
File Transfer	WinSCP	Uploading application files to EC2

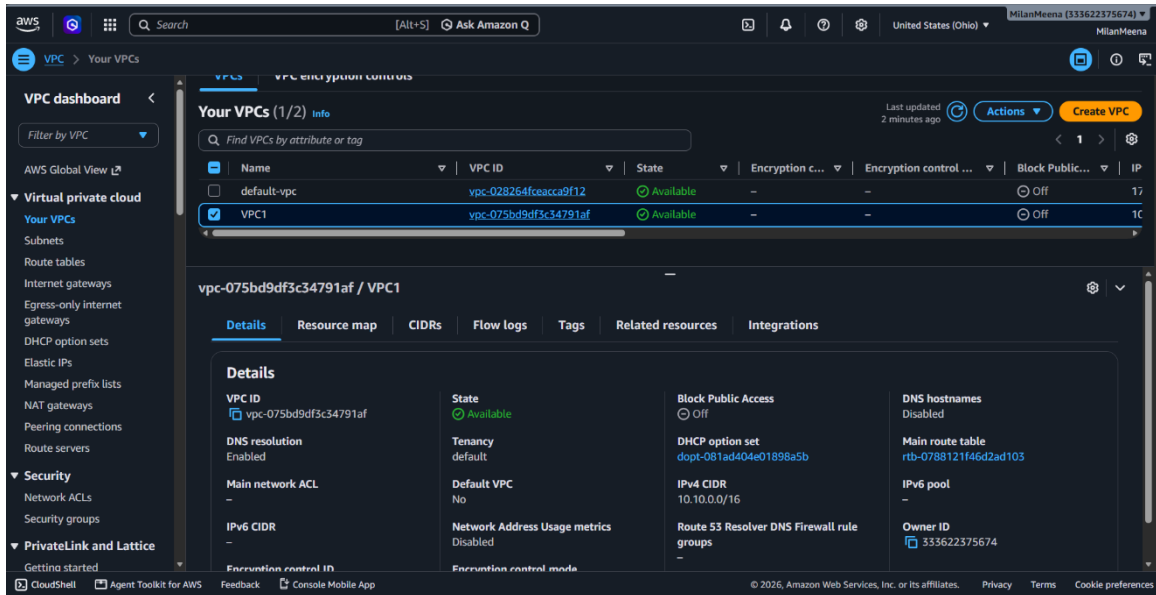
3. Architecture Flow Diagram



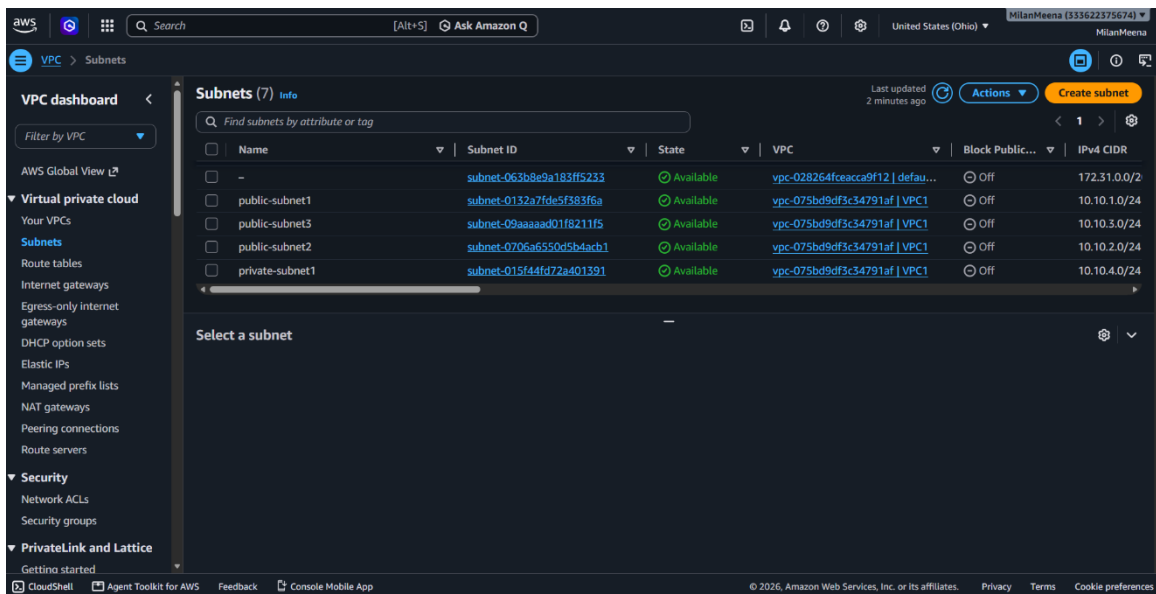
The flow begins with a user request hitting the ALB, which forwards it to one of the healthy EC2 instances managed by the Auto Scaling Group. Each EC2 instance runs Nginx + PHP and serves the QuickLoan frontend. Static images are loaded from S3. Application data (loan submissions) is stored in the RDS MySQL database in the private subnet. The Jump Server acts as the secure SSH entry point for all administrative access.

4. Step 1 - VPC & Networking Setup

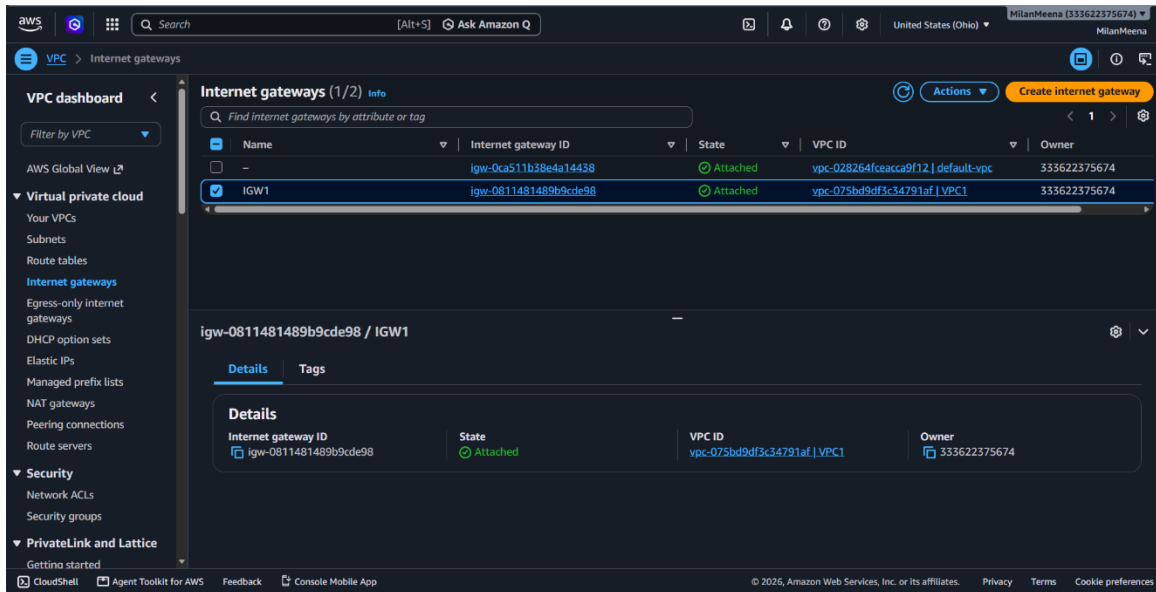
1.1 Create VPC: A custom Virtual Private Cloud (VPC) was created to define an isolated network environment.



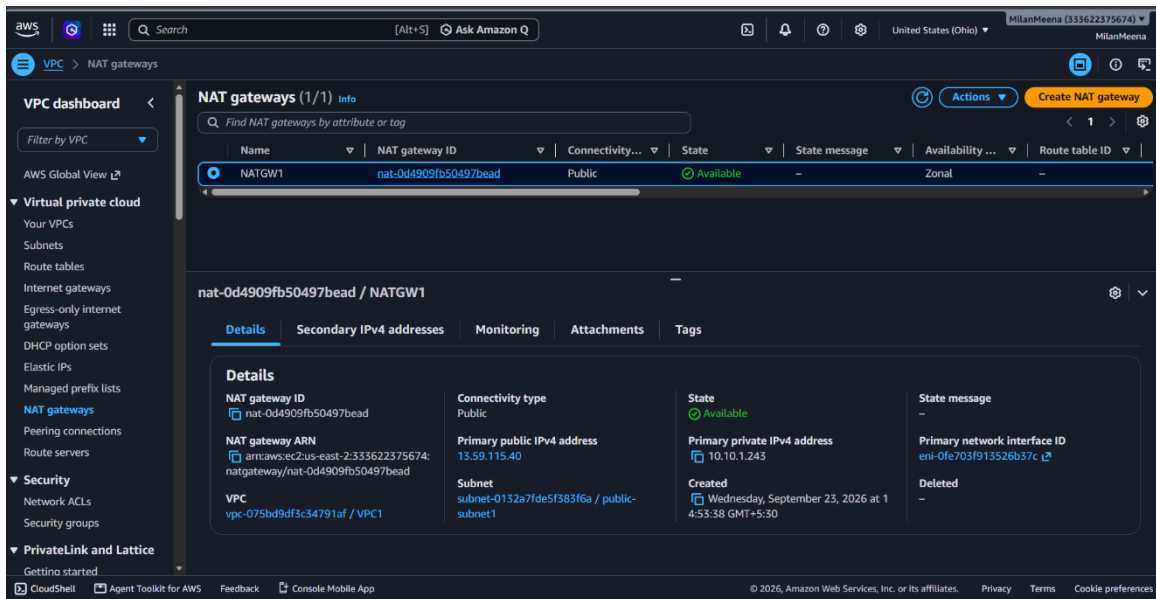
1.2 Create 3 Subnets: Public Subnet 1 (AZ A), Public Subnet 2 (AZ B), and Private Subnet 3 (AZ C).



1.3 Internet Gateway (IGW): Attached to VPC to enable resources in public subnets to communicate with the internet.



1.4 NAT Gateway: Deployed in Public Subnet 1 to allow private instances outbound internet access.



1.5 Route Tables: Configured Public RT (to IGW) and Private RT (to NAT Gateway).

The screenshot displays the AWS Management Console interface for Route Tables. The left sidebar shows the navigation menu with 'VPC' selected. The main content area shows a list of route tables under 'Route tables (1/4)'. The 'Public-RT' is selected, and its details are shown below the list.

Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
-	rtb-05164672418c8274e	-	-	Yes	vpc-028264fca9f12 defau
-	rtb-0788121f46d2ad103	-	-	Yes	vpc-075bd9df3c34791af VPC
Public-RT	rtb-0fcd2cd8e1ae59498	3 subnets	-	No	vpc-075bd9df3c34791af VPC
private-RT	rtb-07925da214b2cfcfd	subnet-015f44fd72a401...	-	No	vpc-075bd9df3c34791af VPC

rtb-0fcd2cd8e1ae59498 / Public-RT

Details

- Route table ID: rtb-0fcd2cd8e1ae59498
- VPC: vpc-075bd9df3c34791af | VPC1
- Main: No
- Owner ID: 333622375674
- Explicit subnet associations: 3 subnets
- Edge associations: -

5. Step 2 - Security Groups Configuration

SG-1 App & Jump Server: Inbound Port 22 (SSH), Port 80 (HTTP).

SG-2 Database Server: Inbound Port 22, Port 3306 (MySQL) restricted to App Server SG.

The screenshot displays the AWS Management Console interface for Security Groups. The left sidebar shows the navigation menu with 'EC2' selected. The main content area shows a list of security groups under 'Security Groups (2/12)'. The 'sg-2-sep-23' is selected, and its details are shown below the list.

Name	Security group ID	Security group name	VPC ID	Description
-	sg-0458471a6561d70a3	ec2-rds-1	vpc-028264fca9f12	Security group a
sg-23-sep	sg-0142d00761a6c4f29	SG1	vpc-075bd9df3c34791af	SG1
-	sg-0c777cdebb7a6e9a3	default	vpc-028264fca9f12	default VPC secu
-	sg-0c8e8081b6f77a366	launch-wizard-8	vpc-028264fca9f12	launch-wizard-8
-	sg-0e5050c8d08c57132	ec2-rds-2	vpc-028264fca9f12	Security group a
sg-2-sep-23	sg-0c566a372deed1a2	SG-2	vpc-075bd9df3c34791af	SG-2
-	sg-046f3c8485feb72bd	default	vpc-075bd9df3c34791af	default VPC secu
-	sg-0f218eb2f949e215d	launch-wizard-3	vpc-028264fca9f12	launch-wizard-3
-	sg-0b1f3ca9b2135b928	launch-wizard-7	vpc-028264fca9f12	launch-wizard-7
-	sg-0cd97a6e0cbf2a2e3	launch-wizard-5	vpc-028264fca9f12	launch-wizard-5

Security Groups: sg-0c566a372deed1a2, sg-0142d00761a6c4f29

6. Step 3 - EC2 Instance Launch

Three EC2 instances launched on Amazon Linux 2023: jump-server (Public), app-server (Public), database-server (Private). All server access was performed using the SSH jump pattern via the Bastion Host.

```
root@ip-10-10-2-189~
login as: ec2-user
Authenticating with public key "jump-keypair"

Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-10-10-1-12 ~]$ sudo su -
[root@ip-10-10-1-12 ~]# vim web-server-keypair.pem
[root@ip-10-10-1-12 ~]# ls -l
total 4
-rw-r--r-- 1 root root 1679 Sep 23 09:49 web-server-keypair.pem
[root@ip-10-10-1-12 ~]# chmod 400 web-server-keypair.pem
[root@ip-10-10-1-12 ~]# ls -l
total 4
-r----- 1 root root 1679 Sep 23 09:49 web-server-keypair.pem
[root@ip-10-10-1-12 ~]# ssh -i web-server-keypair.pem ec2-user@10.10.1.12
The authenticity of host '10.10.1.12 (10.10.1.12)' can't be established.
ED25519 key fingerprint is SHA256:2gmo0JofJ2rGf8Ez91aYthOKiv1a1Em10g.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.1.12' (ED25519) to the list of known hosts.
ec2-user@10.10.1.12: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
[root@ip-10-10-1-12 ~]# ssh -i web-server-keypair.pem ec2-user@10.10.2.189
The authenticity of host '10.10.2.189 (10.10.2.189)' can't be established.
ED25519 key fingerprint is SHA256:VgS83XN7wA2HrIuvy0Xbr2zcczr9tBka5wcc5jFqb34.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.2.189' (ED25519) to the list of known hosts.

Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-10-10-2-189 ~]$ sudo su -
[root@ip-10-10-2-189 ~]# ls -l
[root@ip-10-10-2-189 ~]# chmod 400 web-server-keypair.pem
chmod: cannot access 'web-server-keypair.pem': No such file or directory
[root@ip-10-10-2-189 ~]# exit
logout
[ec2-user@ip-10-10-2-189 ~]$ exit
logout
```

7. Step 4 - Application Deployment (Nginx + PHP)

Installed Nginx and PHP 8.2. Uploaded application files via WinSCP and moved to Nginx web root directory (/usr/share/nginx/html). Set appropriate ownership and permissions.

```
yum update -y

yum install -y nginx

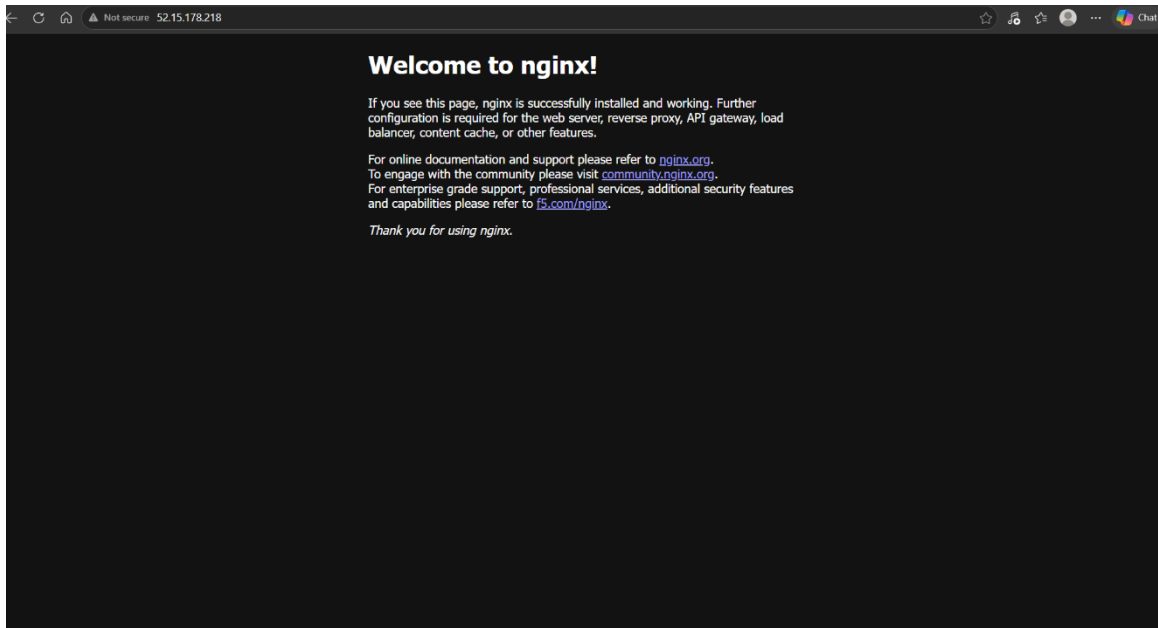
systemctl start nginx

chkconfig nginx on

# Verify: Open browser → http://<public-ip>:80 — should show Nginx default page
```

8. Step 5 - S3 Bucket for Static Assets

Created S3 bucket in us-east-2 to store static image assets. Updated application source files to reference the new S3 bucket URL.

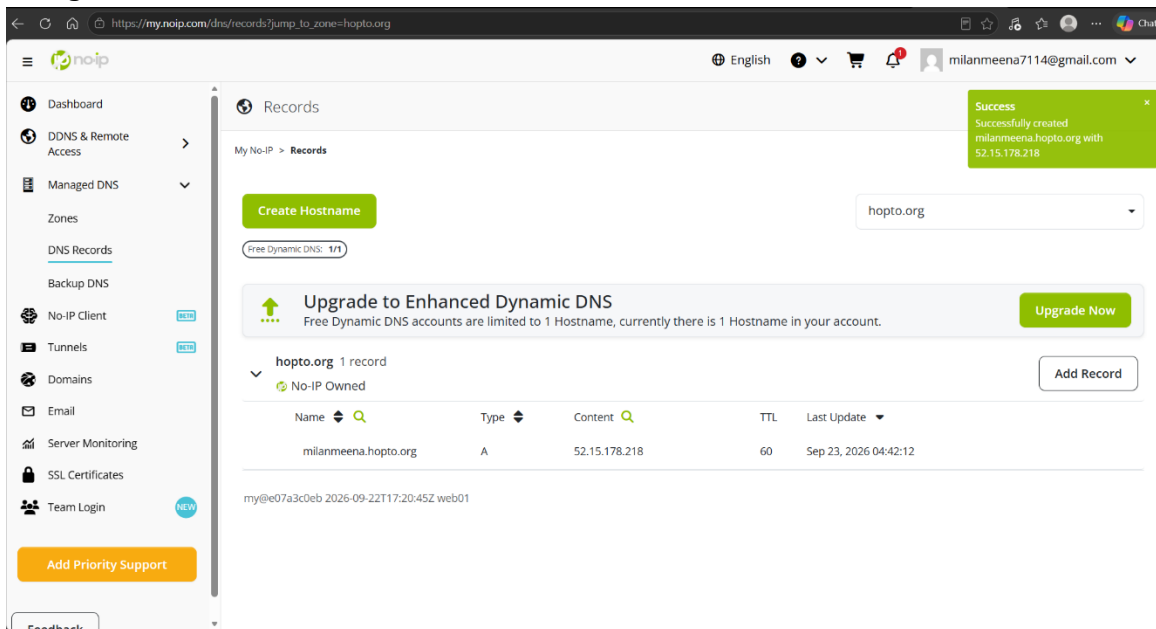


9. Step 6 - RDS MySQL Database Setup

Provisioned RDS MySQL instance in Private Subnet. Updated DB connection configuration on App Server. Created quickloan_db and applications table via Jump Server.

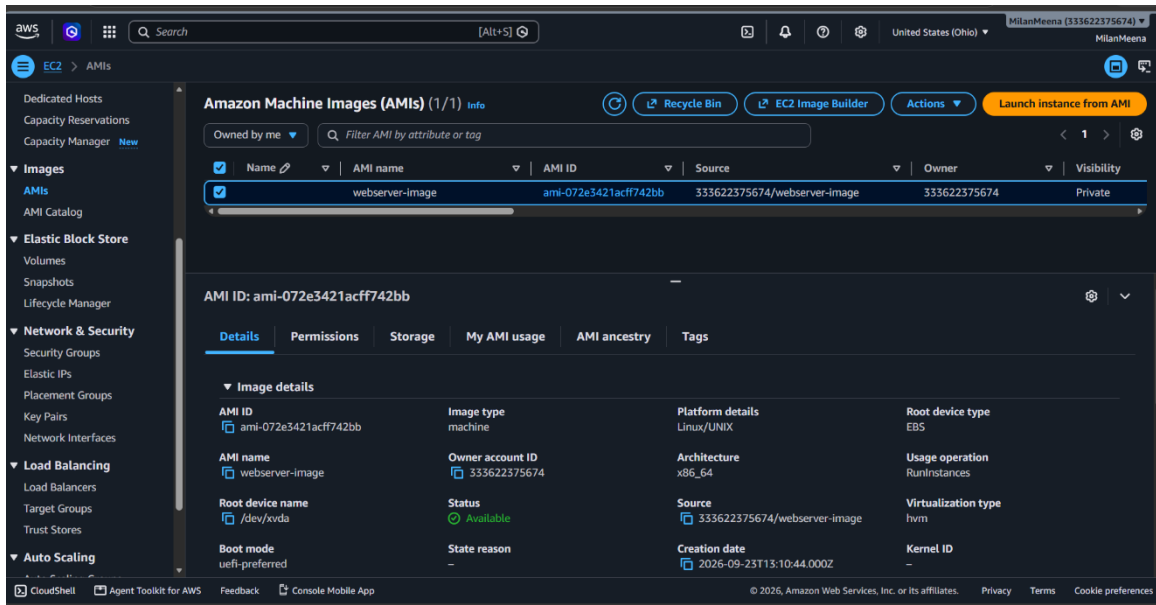
10. Step 7 - Domain Configuration (No-IP)

Registered free dynamic DNS hostname at noip.com. Updated Nginx virtual host configuration to use the domain name.



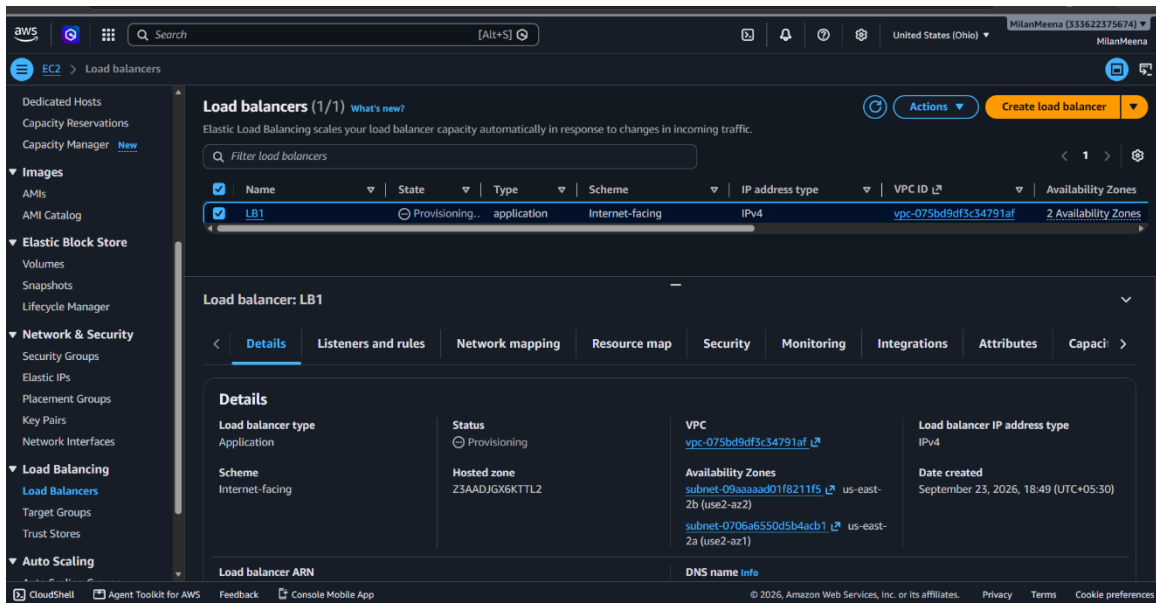
11. Step 8 - AMI Creation

Created custom AMI (app-server-image) from the configured App Server to serve as the golden image for Auto Scaling.



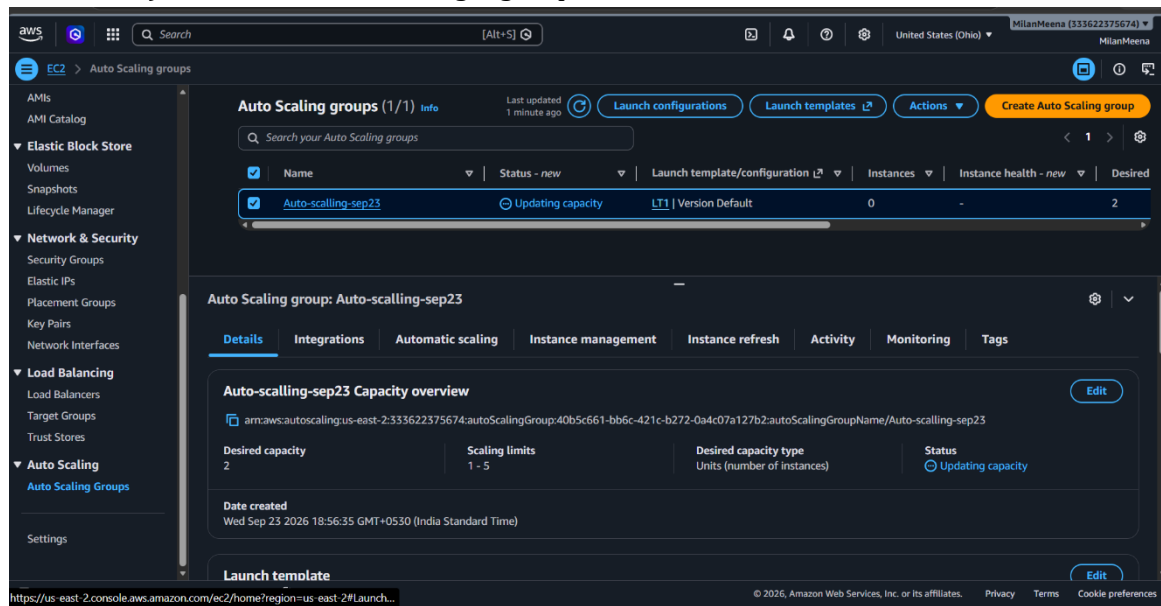
12. Step 9 - Application Load Balancer & Target Group

Created Target Group for HTTP Port 80. Provisioned Internet-facing Application Load Balancer across multi-AZ subnets forwarding to the Target Group.



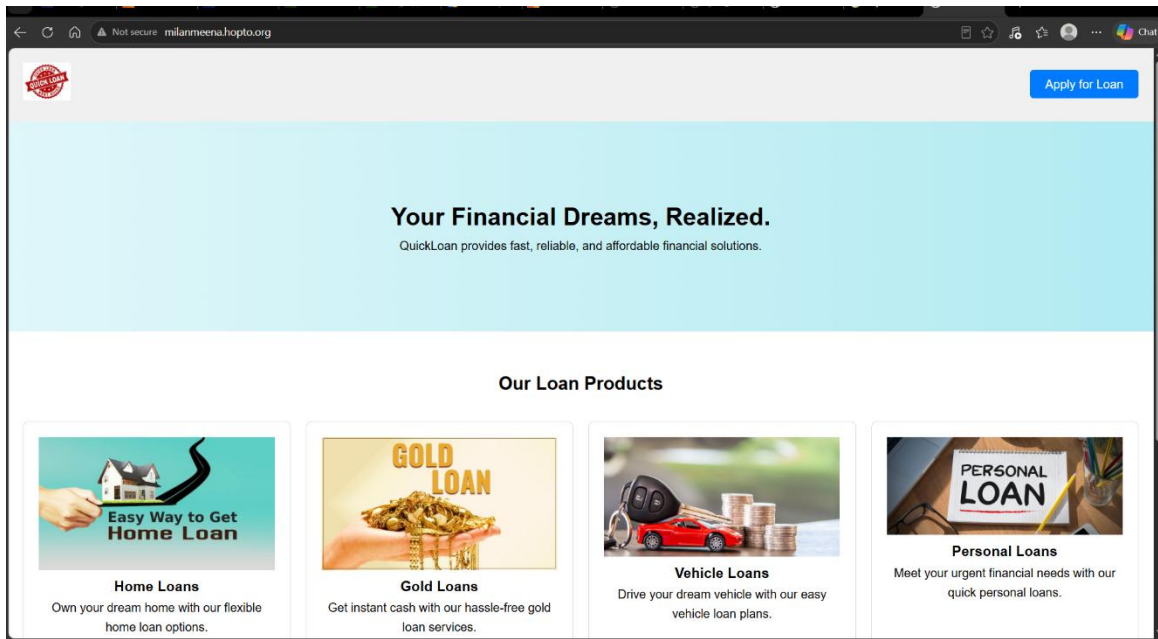
13. Step 10 - Auto Scaling Group Configuration

Configured Launch Template using the custom AMI. Created Auto Scaling Group (Min 2, Max 5 instances) attached to the ALB target group.



14. Step 11 - End-to-End Verification

Application homepage loaded correctly via domain. Submitted test loan application via web form and verified data entry directly in the RDS MySQL database.



15. Roles & Responsibilities

#	Responsibility	Status
1	Designed and created the VPC, subnets, route tables, IGW, and NAT Gateway.	Completed
2	Configured Security Groups to allow necessary traffic (80/443 for EC2, 3306 for RDS).	Completed
3	Deployed and tested PHP app on EC2 with Nginx and PHP-FPM; created custom AMI.	Completed
4	Configured S3 bucket for static asset hosting and updated application source files.	Completed
5	Provisioned Amazon RDS MySQL; created database schema and verified connectivity.	Completed

6	Configured Launch Templates and Auto Scaling Groups for horizontal scaling.	Completed
7	Implemented Application Load Balancer (ALB) with a Target Group for traffic distribution.	Completed
8	Performed end-to-end application testing and database verification.	Completed

16.Database Screenshots

```
Database changed
MySQL [quickloan_db]> select * from applications;
+-----+-----+-----+-----+-----+
| id | name | contact | email | loan_type |
+-----+-----+-----+-----+-----+
| 1 | milan | 1234577890 | milan12@gmail.com | Personal Loan |
+-----+-----+-----+-----+-----+
1 row in set (0.001 sec)
```